

Gestión de memoria

Introducción

Conceptos generales

La gestión de memoria consiste en la asignación de posiciones en una memoria física de capacidad limitada a diferentes procesos que la soliciten con objeto de activarse y entrar en ejecución.

La gestión de memoria debe cumplir las siguientes funciones:

- Determinar la forma de asignación: como se debe dar, a quién y en qué orden.
- Determinar la técnica de desasignación: que procesos han de abandonar la memoria, cuando y en que orden.
- Llevar la contabilidad de las posiciones asignadas y libres.
- Facilitar el aislamiento y protección a los distintos espacios de memoria

En los primeros sistemas no existía en sí una gestión o un orden para dar manejo a la memoria: era responsabilidad del usuario diseñar un orden y configurarlo manualmente. Ya en los sistemas posteriores se dio un orden al mapear la memoria principal y dividirla en dos partes: Una para el sistema operativo (monitor residente, núcleo o kernel) y la otra parte es la “parte de usuario” que es para el programa que se ejecuta en ese instante como sistema monoprogramado. Si el sistema es multiprogramado, esta parte de la memoria debe subdividirse aún más para hacer sitio a varios procesos.

Asignación

Asociando instrucciones y datos a direcciones de memoria



**Código en dispositivo
de almacenamiento**

0100

```
.DATA
    DATO DW    0x1234
```

0200

0202

0204

0206

```
.CODE
    CICLO:
        MOV  1, R1
        ADD  R1, R2
        SUB  R1, [0100]
        JMP  0200
```

Código en memoria

Asignación

Asociando instrucciones y datos a direcciones de memoria

0100

```
.DATA
    DATO DW    0x1234
```

```
.CODE
```

```
    CICLO:
```

0200

```
        MOV    1, R1
```

0202

```
        ADD    R1, R2
```

0204

```
        SUB    R1, [0100]
```

0206

```
        JMP    0200
```

0300

0400

0402

0404

0406

0100

0200

0202

0204

0206

0300

0400

0402

0404

0406

```
.DATA
    DATO    DW    0x1234
```

```
.CODE
```

```
    CICLO:
```

```
        MOV    1, R1
```

```
        ADD    R1, R2
```

```
        SUB    R1, [0300]
```

```
        JMP    0400
```

Máquina desnuda

- Es la manera más sencilla de gestionar la memoria: no existe ningún gestor
- El usuario controla toda la memoria
- El sistema no proporciona ningún servicio

Tiempo de compilación:

- Se conocen a priori las direcciones de memoria
- Código absoluto
- Debe recompilarse si cambia la dirección inicial del programa

Tiempo de carga:

- Código relocizable
- Una vez cargado, no se puede mover de la memoria

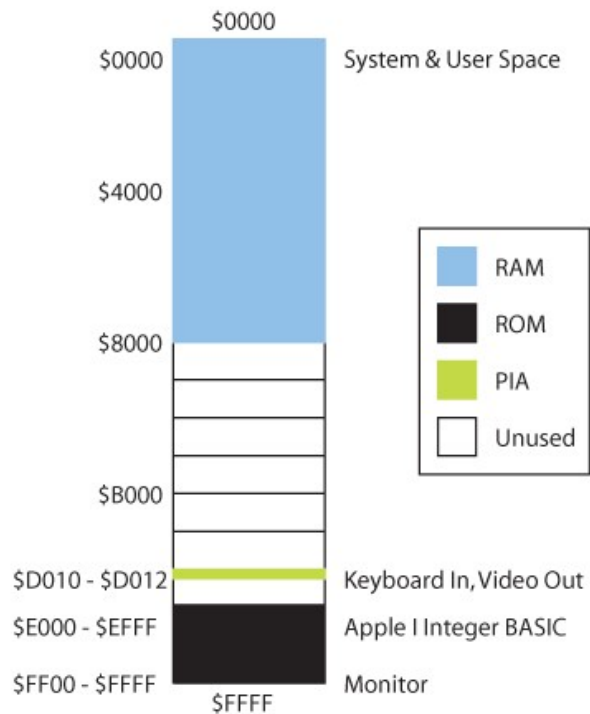
Tiempo de ejecución:

- El proceso puede moverse en la memoria
- La asignación de direcciones se hace hasta el momento de la ejecución
- Se requiere hardware especial

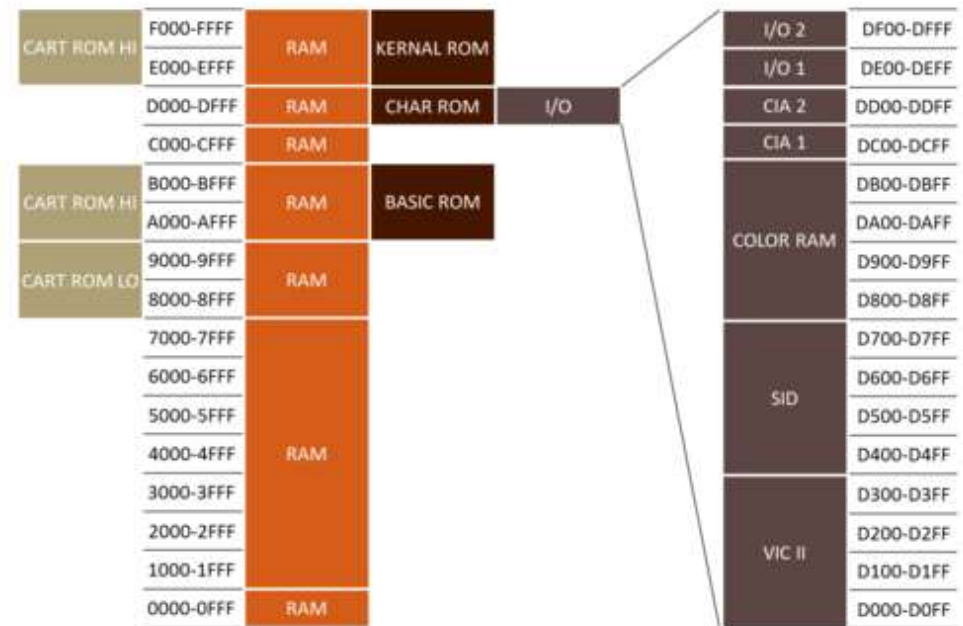
Mapas de memoria

- Un **mapa de memoria** es una estructura de datos que indica cómo está distribuida la memoria en una máquina (arquitectura) y contiene información sobre el tamaño total de memoria y las relaciones que existen entre direcciones lógicas y físicas y proveer otros detalles específicos sobre la arquitectura del computador.
- Generalmente se crean en el proceso de arranque por el firmware (BIOS, UEFI) e indica al sistema operativo cómo está distribuida (diseño) la memoria.
- Cada área específica de memoria (rango de direcciones) puede ser asociada a:
 - Dispositivos: donde podemos comunicarnos para leer o escribir con el dispositivo asociado.
 - Sistema operativo: para llamados al núcleo (kernel) del sistema, trampas e interrupciones.
 - Proceso de usuario: que corresponden al un programa ejecutado por el usuario. (o varios si el multiprogramado)

Mapas de memoria



Apple I



Commodore 64

Gestión de memoria

Asignación de memoria en sistemas monoprogramados

Asignación en sistemas monoprogramados

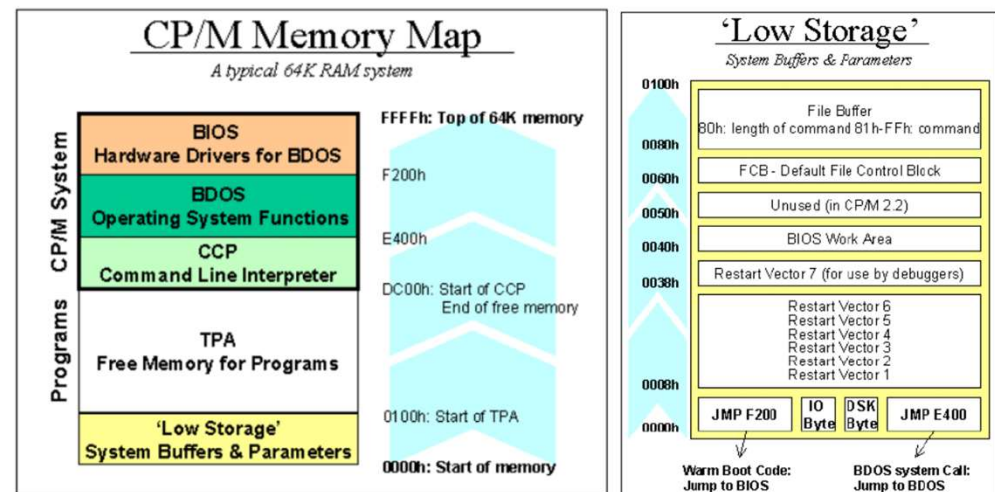
En un sistema monoprogramado, la memoria principal se divide en dos partes contiguas:

- Una para el sistema operativo (núcleo o kernel) que ha de residir permanentemente en la memoria (**monitor residente**).
- La otra parte es la “parte de usuario” que es para el programa que se ejecuta en ese instante.

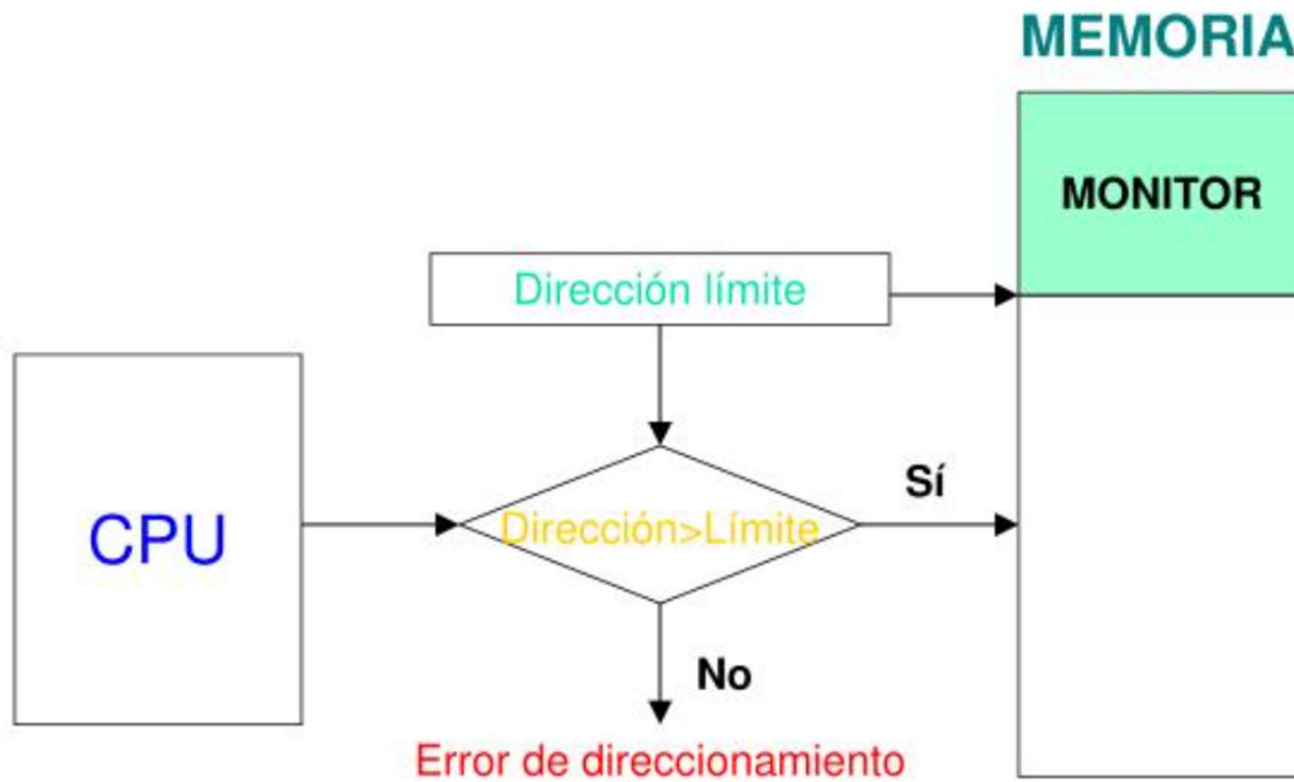
Para poder suministrar un espacio contiguo, generalmente se localiza en un extremo de la memoria el área del sistema operativo.

Siempre que se carga el proceso se asegura que sus direcciones estén dentro de los límites que corresponden a la región de usuario. Si la dirección resulta por fuera de esta área, el sistema operativo provoca una interrupción y el proceso no es cargado en memoria.

Esta dirección límite a veces se especifica por hardware y en otros casos se utiliza un registro límite o base donde se graba la dirección de comienzo de la zona del usuario.



Monitor residente

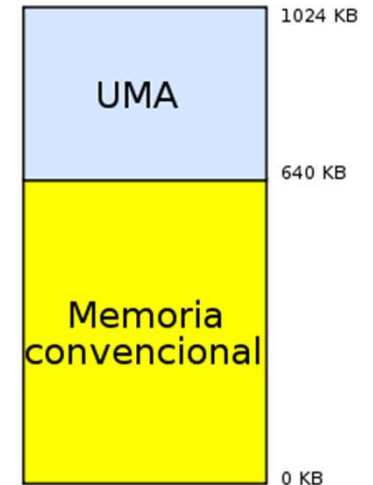


Sistemas monoprogramados : MS-DOS

Los primeros procesadores de Intel para PC (8086/8088) utilizaban 20 bits para acceder a la memoria, por lo que podía direccionar 2^{20} bytes = 1 MiB*.

IBM decidió que los primeros 640K serían para el usuario, para que ejecutara sus aplicaciones, compartiendo esta memoria con el núcleo del sistema operativo y los drivers de dispositivos que fuera instalando.

El área de memoria superior o UMA (Upper Memory Area) que son los 384K restantes se los reservó IBM para futuras mejoras hardware del equipo, aquí incluyó el software controlador de vídeo. Esta es la característica que crea la barrera de los 640 kilobytes (KB).



IBM reservó la región más alta del mapa de memoria de la PC para la memoria ROM del BIOS (8 KiB en una dirección justo abajo del megabyte), además, memoria RAM para los periféricos como las tarjetas de video CGA y MDA, otras ROM del sistema, y entrada/salida (E/S) mapeada en memoria. En el caso de las computadoras fabricados por IBM, también había un interpretador BASIC, escrito por Microsoft que ocupaba 40 KiB por debajo del BIOS.

¡Y se nos acabó el mega!

* ¡una barbaridad para la época! Ninguno de los computadores de uso doméstico en 1981 (Apple II, Atari, Spectrum, Commodore 64, SpectraVideo, ...) llegaban a los 64 KiB

0x110000	Area alta de memoria	
0x100000	Area de memoria extendida (XMS)	
0x0e0000	Sistema ROM BIOS	
0x0d0000	Area de intercambio de la memoria extendida (EMS)	
0x0a0000	Bloques de memoria alta sin usar	
0x0a0000	Memoria de Video VGA	
0x000600	Espacio de usuario	
0x000400	Shell (línea de comandos)	<i>command.com</i>
	Rutinas de DOS	<i>msdos.sys</i>
	Manejador de Dispositivos	<i>io.sys</i>
0x000000	Tablas del ROM BIOS	
0x000000	Tabla del Vector de Interrupciones	

➔ DOS

Cuando los programas se fueron reclamando más memoria (p.ej. hojas de cálculo gigantescas), se encontraron con la limitación de los 640 KiB. Se hizo necesario ampliar la memoria RAM por encima del MiB, inaccesible para **MS-DOS**. Como respuesta a estas limitaciones de memoria del MS-DOS, surgieron:

- **Memoria expandida - EMS: expanded memory specification- (puede llegar a los 64 MB):** En la UMA siempre quedaban algunas zonas que no eran utilizados por nadie. Se permitió instalar en los equipos unos chips de ampliación de memoria independientes (que no podían tener espacio de direcciones, pues el procesador ya tenía su 1 MiB completamente direccionado). Cuando un programa se ejecuta, en vez de residir en el disco se encuentra cargado en la memoria expandida, y cuando el procesador necesita informaciones contenidas en esta memoria, un controlador vuelca el contenido a zonas que queden libres en la memoria superior a donde sí llega el procesador (vuelca bloques de 16 KiB).

El procesador debe funcionar en **modo real**, y es necesario cargar un controlador especial (EMM386.EXE: expanded memory manager). Es decir, la memoria expandida cumplía la función que ahora tiene la memoria virtual, era una zona para descargar las partes de la aplicación que no eran necesarias en un momento dado.

- **Memoria extendida -XMS: extended memory specification- (por encima del Mega).** No puede ser utilizada directamente por el MS-DOS, ya que éste sólo reconoce direcciones de 0 a 1024 KiB, aunque el microprocesador 80286 podía acceder a 16 MiB, y el 80386 con un bus de 32 bits llegaba a los 4 GiB. El sistema operativo de IBM, OS/2, si era capaz de aprovechar esta memoria.

Para acceder desde DOS necesitamos de unos controladores de dispositivo especiales (como RAMDRIVE.SYS que crea discos en la memoria RAM). A los primeros 64 KiB de memoria extendida se los conoce como área de memoria alta (HMA), que a partir de MS-DOS 5.0 se pueden gestionar. Suele utilizarse para instalar controladores que antes iban en memoria convencional (comando LH -loadhigh-), así queda más espacio para el usuario.

El comando MS-DOS para visualizar la cantidad de memoria usada y libre por el sistema es:

MEM

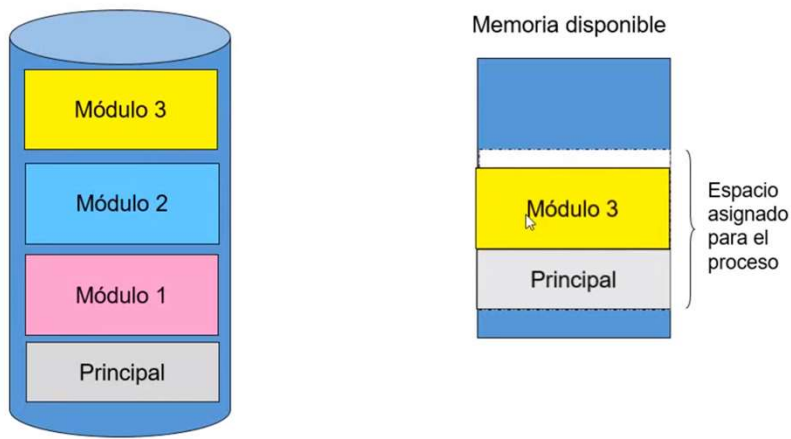
Tipo de memoria	Total = Usada + Libre		
-----	-----	-----	-----
Convencional	640K	28K	612K
Superior	111K	82K	29K
Reservada	384K	384K	0K
Extendida (XMS)	7,057K	6,221K	836K
-----	-----	-----	-----
Memoria total	8,192K	6,715K	1,477K
Total menor 1 MB	751K	110K	641K

modo real: una tarea puede acceder a cualquier lugar de la memoria.

modo protegido: Los procesos tienen establecidos unos límites de memoria de los que no pueden salir. Desde el 80286.

Carga dinámica

También conocida como **Overlay** o **superposición**, significa transferir un bloque de código de programa o datos a la memoria principal, reemplazando lo que ya está almacenado en ella.



- Permite que los programas sean más grandes que la memoria principal de la computadora.
- Una rutina o módulo no se carga hasta que es llamada.
- Una rutina no usada nunca se carga, permitiendo una mejor utilización del espacio en memoria.
- Útil cuando grandes cantidades de código se necesitan para manejar casos que no ocurren frecuentemente.
- No se requiere un soporte especial del sistema operativo, ya que puede implementarse por diseño del programa.

Gestión de memoria

Modelos de memoria: asignación de memoria en sistemas multiprogramados

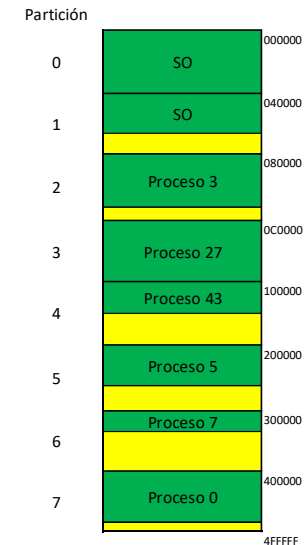
Objetivos del sistema de gestión de memoria

En un sistema con **multiprogramación** se deben cumplir los siguientes principios:

- Reubicación
- Protección
- Compartición
- Mantener la organización lógica de los programas
- Conocer su organización física
- Maximizar el rendimiento

Particiones estáticas

- La memoria se divide en cierto número de **particiones**, cada una de las cuales contendrá un proceso.
- La dirección base de cada proceso es la dirección de inicio de cada partición.
- El tamaño de las particiones (que puede ser uniforme o variable) es un parámetro del Sistema Operativo, por ejemplo:
 - Para una memoria de 2 MiB, con particiones fijas de 256 KiB se tendrían 8 bloques.

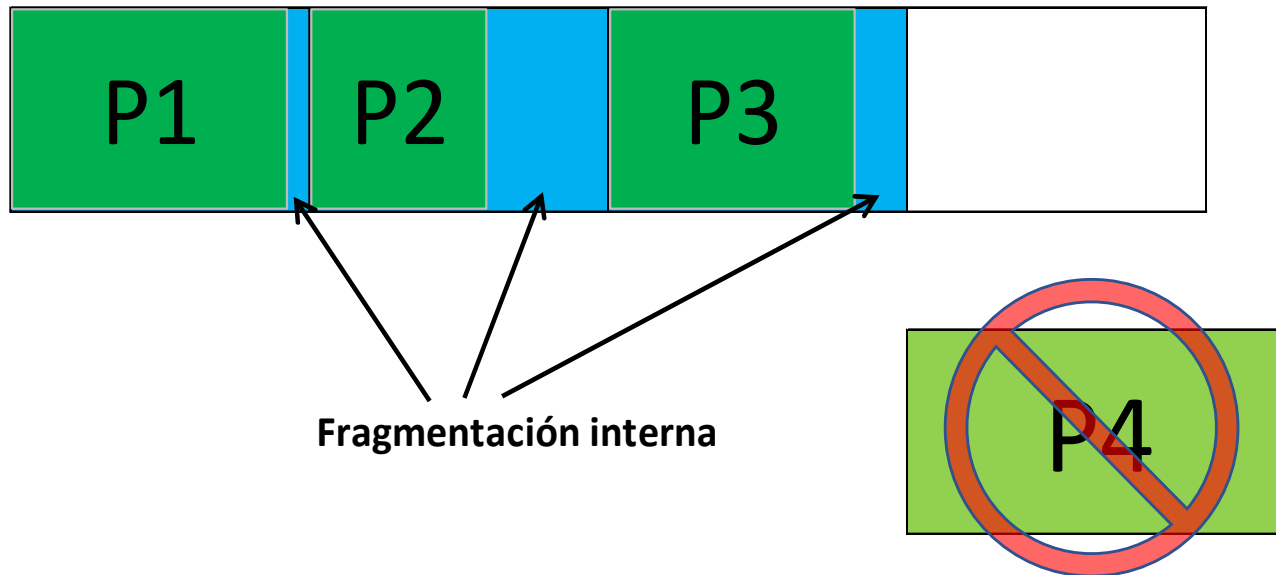


Espacio particionado
 Espacio libre

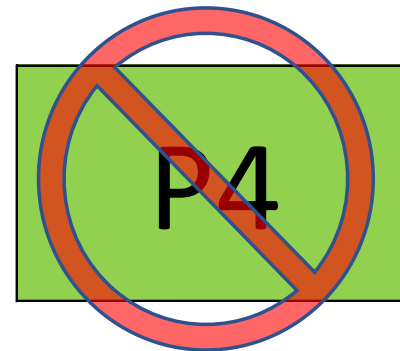
PID	L/O	Estado	Dirección BCP
SO	0	Listo	04B000
SO	0	Listo	04B100
P03	0	Activo	04B200
P27	0	Listo	04B300
P43	0	Listo	04B400
P05	0	Bloqueado	04B500
P07	0	Bloqueado	04B600
P00	0	Listo	04B700

- El SO mantiene una **tabla de descripción de particiones** (procesos) en la que cada registro corresponde a una partición, conteniendo información como:
 - Identificación del proceso (PID), si está libre o no, estado, dirección en memoria del bloque de control de proceso (BCP).

Particiones estáticas de tamaño fijo



Particiones estáticas de tamaño variable



Particiones dinámicas

- Los programas son introducidos por el SO inicialmente en posiciones consecutivas de memoria, no existiendo por tanto particiones predefinidas.
- Gestión:
 - Tabla de procesos, en la que cada registro contiene el PID, dirección base, capacidad que ocupa, estado, dirección del BCP, etc.
 - Tabla de fragmentos libres, donde el planificador de trabajos introduce en la memoria principal los procesos que quepan en los fragmentos libres y luego actualiza las dos tablas.

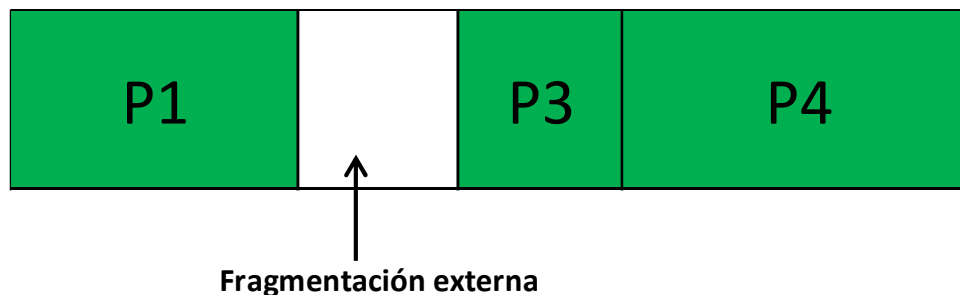


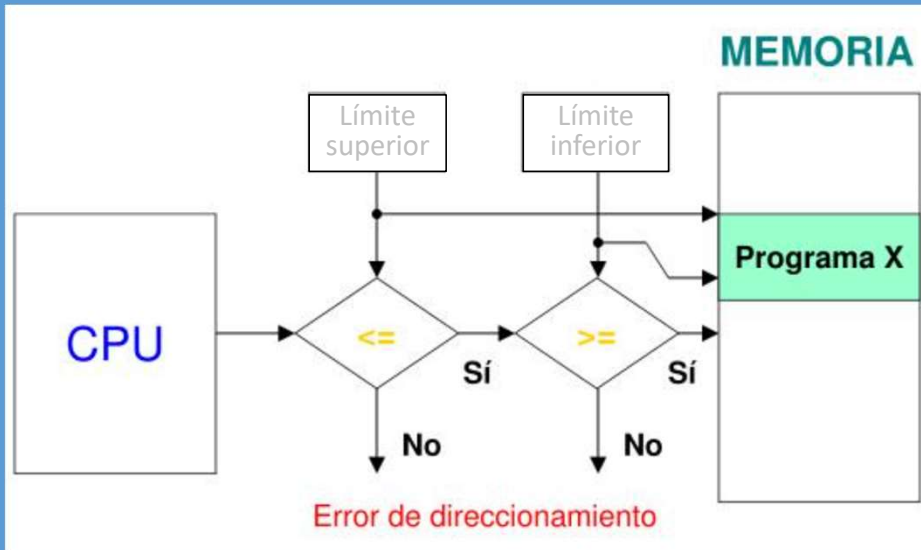
Tabla de descripción de párticiones				
PID	Dirección base	Capacidad (KiB)	Estado	Dirección BCP
SO	000000	256	Listo	04B000
SO	040000	200	Listo	04B100
P03	077000	163	Activo	04B200
P27	09FC00	256	Listo	04B300
P43	0DFC00	126	Listo	04B400
P05	0FF400	110	Bloqueado	04B500
P07	11AC00	97	Bloqueado	04B600
P00	133000	210	Listo	04B700
P06	167800	415	Listo	04B800

Tabla de fragmentos libres

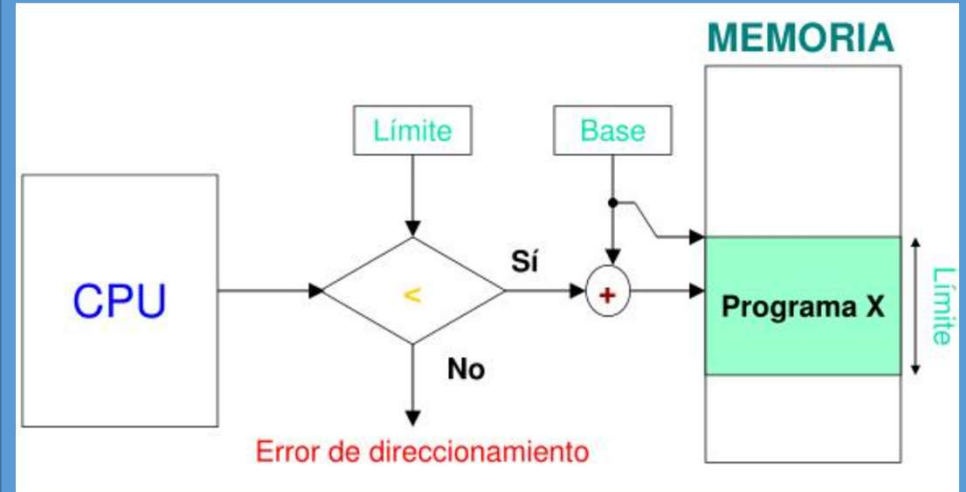
Dirección base	Capacidad (KiB)
1CF400	315

Gestión de las particiones: asignación

Modelo Registros límite
(No soporta la reubicación dinámica)

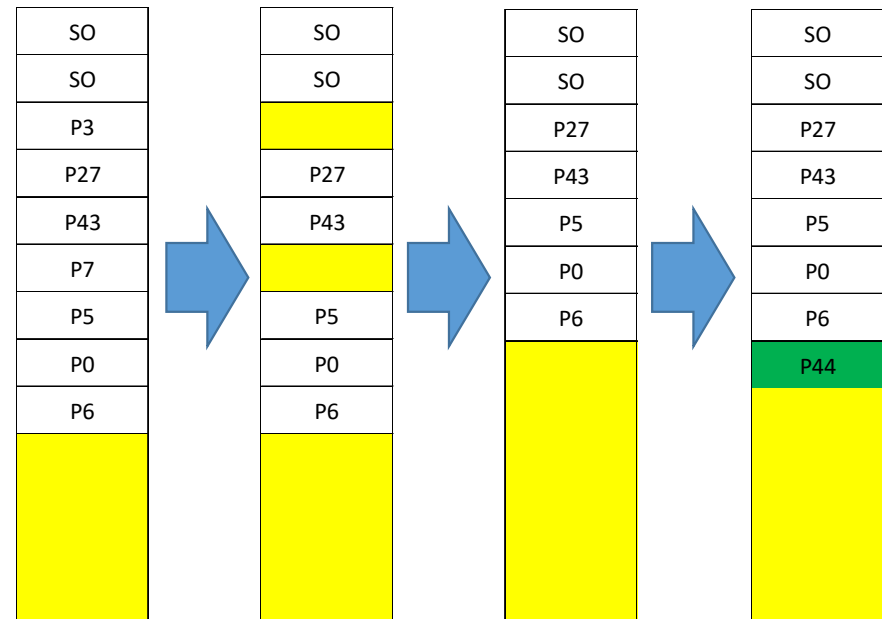


Modelo Registros base-límite
(Soporta la reubicación dinámica)



Reubicación de procesos: compactación

- Cuando se acaba un proceso deja un **fragmento libre** que solo puede ser ocupado por otro proceso de igual o menor capacidad.
- El problema es que al ir acabando de ejecutarse los programas, el número de fragmentos libres crecerá (**fragmentación externa**), llegando un momento en que el porcentaje de memoria aprovechado es muy reducido.
- El problema se resuelve haciendo una **compactación** que agrupe todos esos fragmentos libres, **reubicando** los programas en ejecución.
- Una vez unidos todos los fragmentos libres, existe la posibilidad de introducir nuevos procesos.

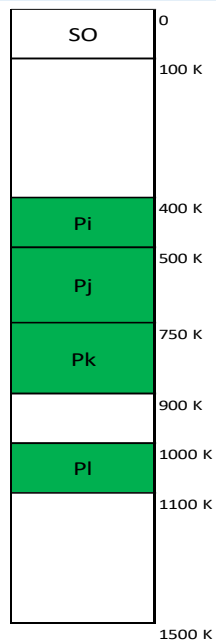


Asignación de memoria con particiones

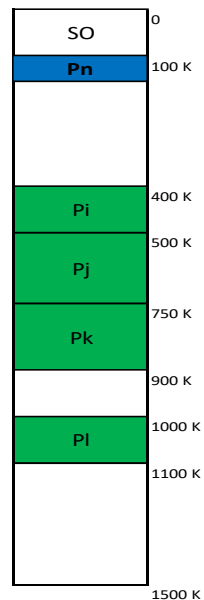
Para la selección de un **área libre en la memoria** y crear una partición para asignarla a un proceso, se determina con uno de los siguientes algoritmos habituales:

- **Primer ajuste:** Asignar el primer fragmento libre que tenga el tamaño suficiente.
- **Mejor ajuste:** Asignar el fragmento más pequeño que tenga el tamaño suficiente.
- **Peor ajuste:** Asignar el fragmento más grande.

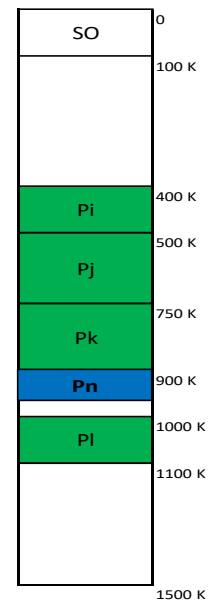
Ejemplo: A partir del siguiente mapa de memoria, debemos ubicar el proceso $P_n = 70$ KiB usando los algoritmos mencionados.



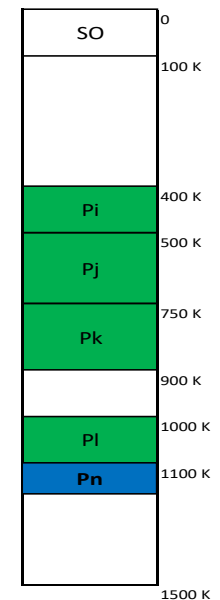
Primer ajuste



Mejor ajuste



Peor ajuste



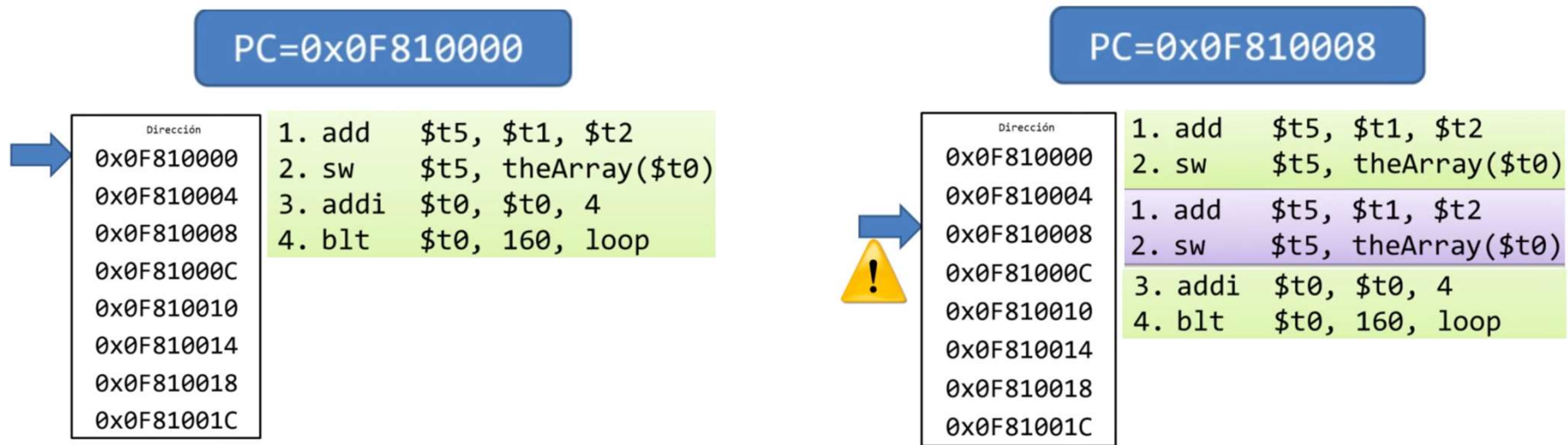
Resumen: Memoria particionada contigua

- Se asigna una partición de memoria a cada proceso

Memoria con partición fija	Memoria con partición variable
Sistema multiprogramado con tamaño y número de particiones fijo	Sistema multiprogramado con tamaño y número de particiones variable
Fragmentación interna y externa	Fragmentación externa
Desventaja en la asignación dinámica de memoria	Ventaja en la asignación dinámica de memoria
Problema: intercambio de trabajos con E/S pendiente	Requiere algoritmos de gestión más complejos

- Fragmentación externa: compactación.

Memoria no contigua: necesidad de instrucciones secuenciales



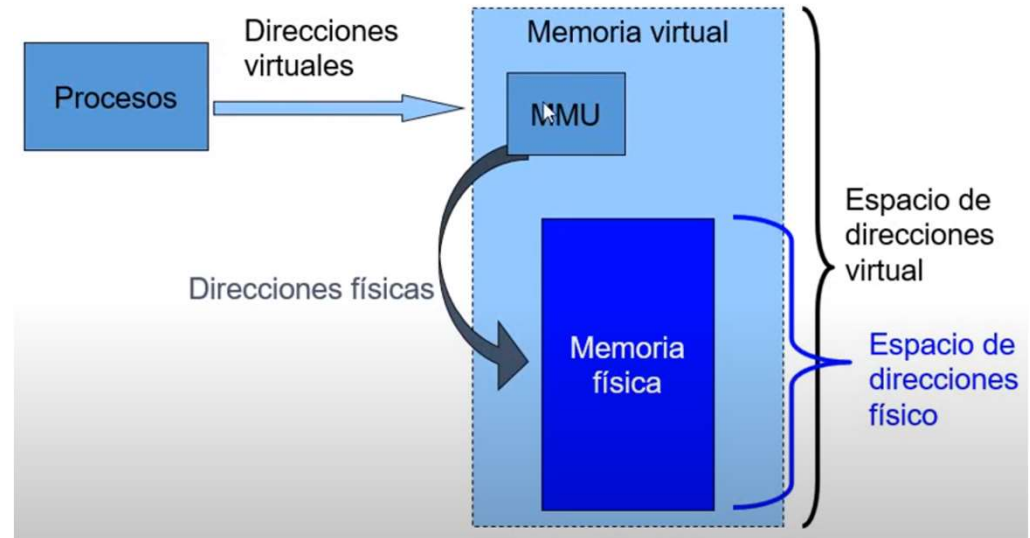
Espacio de direcciones lógico vs. físico

Dirección física

- Cada una de las posiciones de la memoria principal.
- En las arquitecturas actuales, existe un IC denominado MMU que tiene acceso a estas posiciones y sirve de “traductor”.

Dirección lógica

- Se refiere a una posición virtual, para solucionar la secuencialidad en los datos de un proceso.
- En las arquitecturas actuales, son generadas por la CPU



MMU (Unidad de manejo de memoria): Es un dispositivo de hardware que MAPEA las direcciones virtuales a direcciones físicas. Usa un registro de relocalización, que se añade a cada dirección generada por un proceso de usuario a la vez que se envía a memoria.

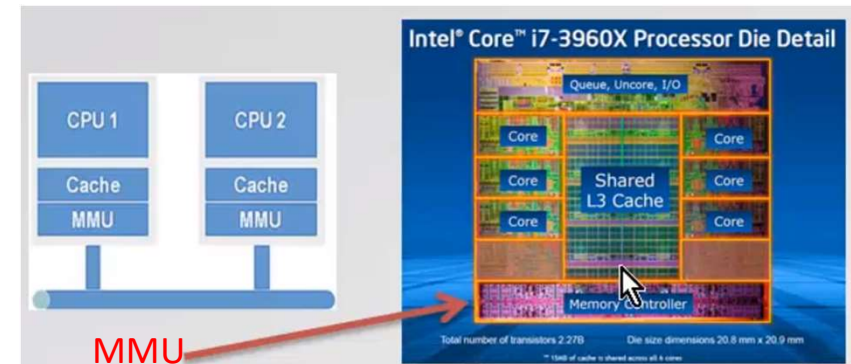
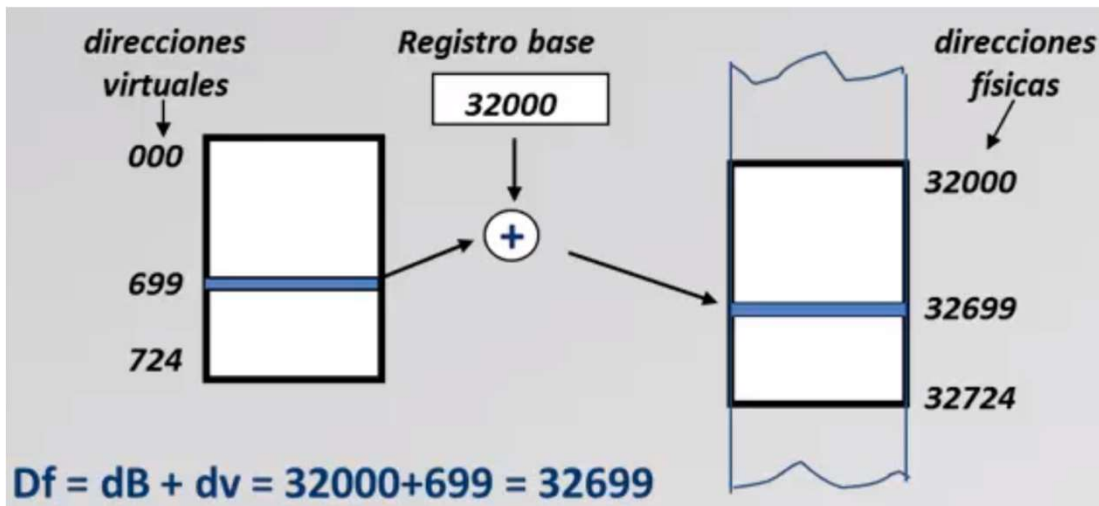
El programa de usuario trata con direcciones lógicas (virtuales). Nunca ve el espacio de direcciones físicas.

Los programas se cargan en la memoria principal a partir de una posición de memoria, en función de las zonas libres, denominada **dirección base (DB)**. Las direcciones reales donde se carga el programa se denominan **direcciones físicas (DF)**.

Espacio de direcciones lógico vs. físico

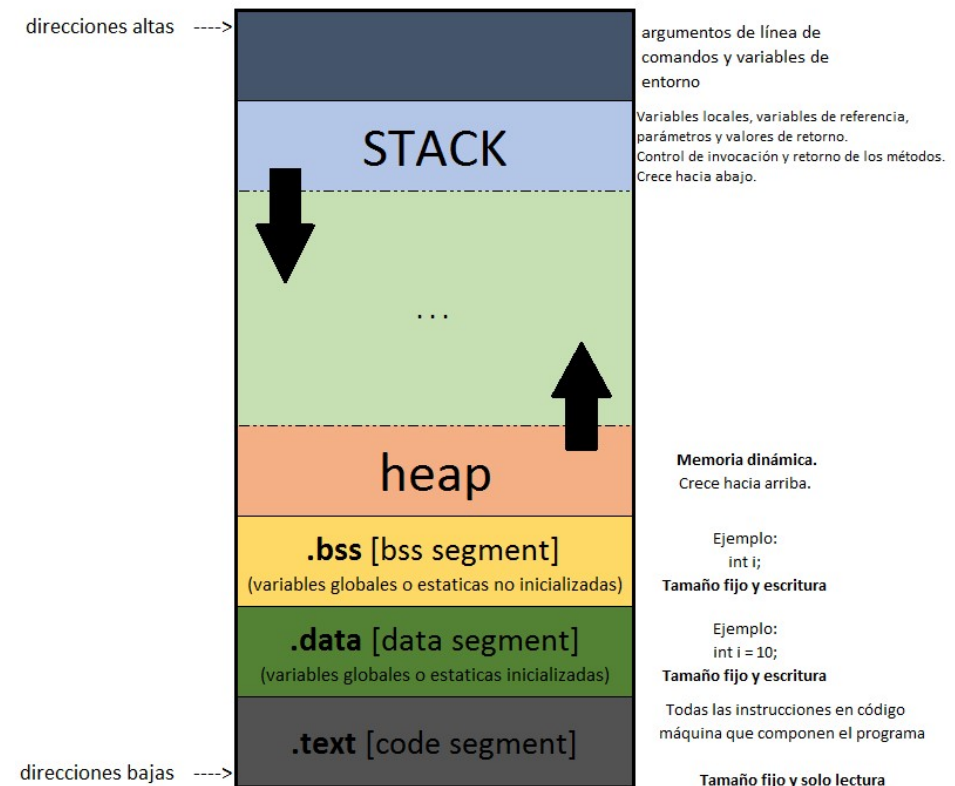
Para hacer la transformación de direcciones virtuales a físicas:

- Se suele realizar con la ayuda de un registro base (que se carga previamente en la dirección base) que se encuentra en la propia CPU o en la MMU.
- En la actualidad, la MMU suele incluirse en el mismo chip del procesador y realiza estas transformaciones (anteriormente realizadas por software).



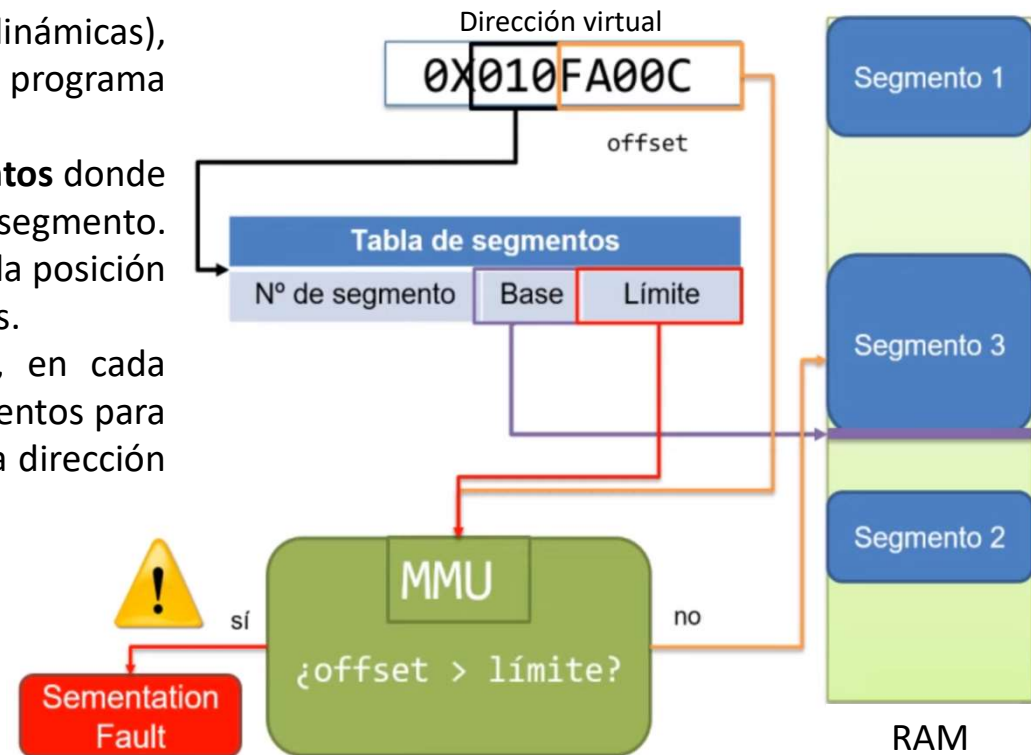
Segmentación

- La memoria se aprovecha tanto mejor cuanto más pequeños sean los procesos o **unidades** a almacenar en ella.
- La gestión de memoria con segmentación consiste en considerar como unidades de almacenamiento **segmentos**, en lugar de procesos.
- Un **proceso** está compuesto por unidades denominados segmentos.
 - **Segmento de código**: que almacena el código ejecutable del proceso en lenguaje de máquina.
 - **Segmentos de datos**: Que es el conjunto de datos, vectores, objetos, etc.
 - **Segmento de pila**.
- **Solapamiento**: La segmentación permite que ciertos procesos puedan compartir código (rutinas, etc.) o datos comunes sin necesidad de estar duplicados en la memoria (segmentos reentrantes).
- Cuando un proceso requiere más memoria se crea un nuevo segmento.



Segmentación: Gestión

- La gestión la realiza el SO (como con las particiones dinámicas), sólo que cada partición en vez de corresponder a un programa completo, corresponde a un segmento.
- El SO mantiene por cada proceso una **tabla de segmentos** donde se especifica su capacidad y la dirección base de cada segmento. En la **tabla de procesos** hay un campo donde se indica la posición de memoria donde se encuentra su tabla de segmentos.
- Para transformar las direcciones virtuales a físicas, en cada acceso de memoria la MMU consulta la tabla de segmentos para ver si la dirección sobrepasa el límite y para obtener la dirección base correspondiente.
- Protección de memoria:** Las principal ventaja del uso de segmentación es que nos permite que cada uno de los segmentos tenga un distinto juego de permisos: 3 bits a la tabla de descriptores de segmentos para los permisos (r w x).
- Solapamiento:** Se puede hacer que 2 segmentos se superpongan de manera que compartan direcciones de memoria física con direcciones lógicas diferentes. Para evitar problemas de concurrencia, debe indicarse explícitamente que una porción de memoria pueda ser compartida. De esta manera, procesos diferentes pueden compartir información y código usando la memoria común.



```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
```

```
void segfunc(int signal, siginfo_t *si, void *arg)
```

```
[shellwave@skylab c085]$ gcc -o foo foo.c
[shellwave@skylab c085]$ ./foo
Segmentation fault (core dumped)
[shellwave@skylab c085]$
```

```
colln@Spectre:~$ Graphileon
Segmentation fault (core dumped)
colln@Spectre:~$ gdb Graphileon
GNU gdb (Ubuntu 9.1-0ubuntu1) 9.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from Graphileon...
(No debugging symbols found in Graphileon)
(gdb) run
Starting program: /usr/local/bin/Graphileon
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
0x0000000000a4fa30 in ?? ()
```

```
(gdb) backtrace
```

```
#0 0x0000000000a4fa30 in ?? ()
#1 0x00007ffff7d802b2 in node::http2::Http2Session::Callbacks::Callbacks(bool) () from /opt/Graphileon/
#2 0x00007ffff7d80375 in ?? () from /opt/Graphileon/libnode.so
#3 0x00007ffff7fe0b8a in ?? () from /lib64/ld-linux-x86-64.so.2
#4 0x00007ffff7fe0c91 in ?? () from /lib64/ld-linux-x86-64.so.2
#5 0x00007ffff7fd013a in ?? () from /lib64/ld-linux-x86-64.so.2
#6 0x0000000000000001 in ?? ()
#7 0x00007ffff7ffe323 in ?? ()
#8 0x0000000000000000 in ?? ()
```

```
(gdb)
```

MS_WebcheckMonitor: explorer.exe - Application Error



The instruction at 0x59a783ff referenced memory at 0x00000000. The memory could not be read.

Click on OK to terminate the program

OK

CraftWare



Fatal error: Unexpected exception!
Invalid memory access (segmentation fault)

Details have been written to file:
C:/Users/Josh/AppData/Local/Temp/CWError.txt
Please send it to support for inspection.

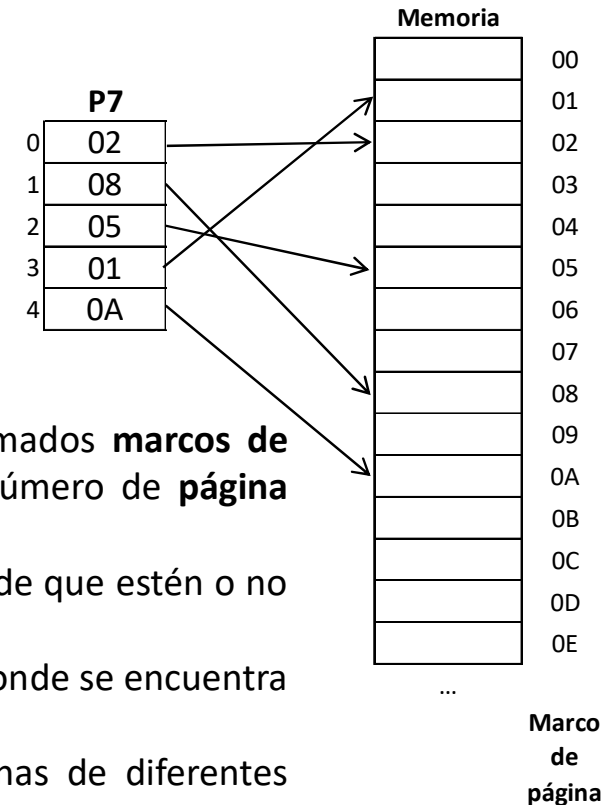
OK

Paginación

- El fundamento está en que no es necesario que el programa completo se almacene en posiciones consecutivas de la memoria principal, pero para el proceso en sí todas las posiciones son consecutivas en posiciones virtuales.
- Se dividen los programas en pequeñas partes o **páginas lógicas**.

Tabla de páginas - P7

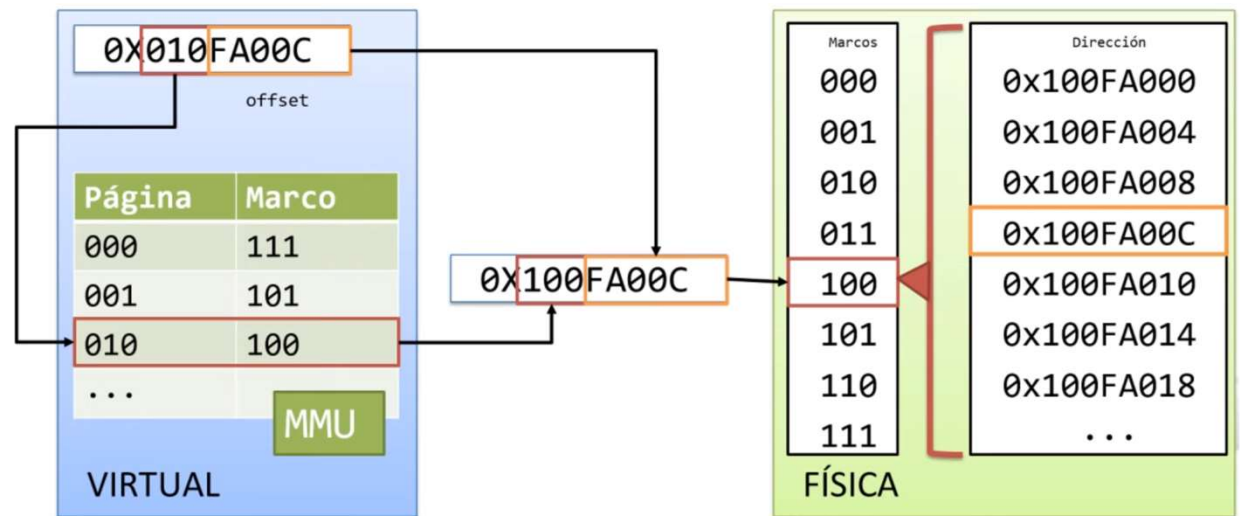
	Página	Marco
	0	02
	1	08
	2	05
	3	01
	4	0A



- La memoria es dividida en trozos del mismo tamaño que las páginas llamados **marcos de página**. Los marcos están identificados por un número, conocido como número de **página física**. Cada página física se asigna en exclusividad a un proceso.
- Las páginas se almacenan en marcos de página libres, independientemente de que estén o no contiguos.
- El SO mantiene por cada proceso una tabla de páginas que indica el marco donde se encuentra cada página del proceso.
- En un momento cualquiera, la memoria se encuentra ocupada con páginas de diferentes procesos, mientras que algunos marcos están disponibles para su uso. El sistema operativo mantiene una lista de estos últimos marcos.
- La cantidad de memoria desperdiciada por un proceso es el final de su última página, lo que minimiza la fragmentación interna y evita la externa.

Paginación: Gestión

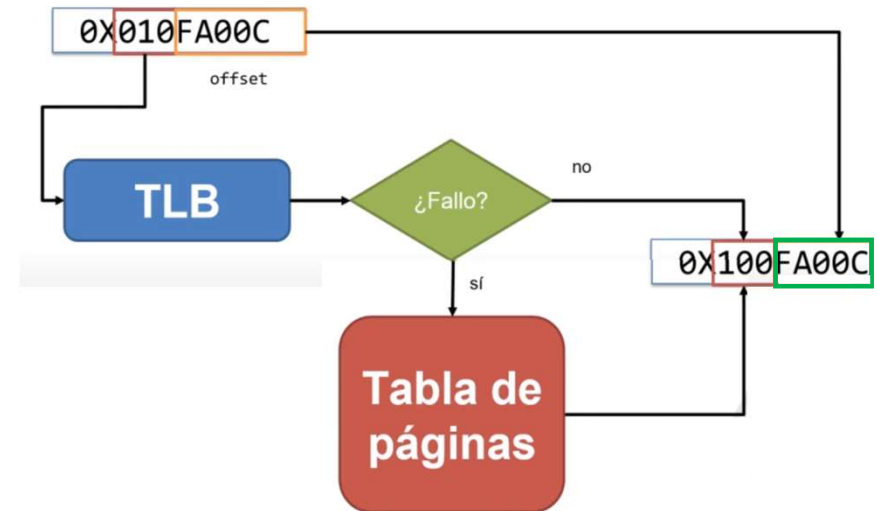
- En la tabla de páginas de un proceso, se encuentra la ubicación del marco que contiene a cada una de sus páginas.
- Las direcciones lógicas ahora se forman como un número de página y de un desplazamiento dentro de esa página (conocido comúnmente como offset).



- El número de página es usado como un índice dentro de la tabla de páginas, y una vez obtenida la dirección del marco de memoria, se utiliza el desplazamiento para componer la dirección real o dirección física.
- En definitiva, para obtener la dirección física es suficiente con concatenar el código de marco de página con el desplazamiento.

TLB – Translation Lookaside Buffer

El **búfer de traducción anticipada** es una memoria caché administrada por la unidad de gestión de memoria (MMU), que relaciona las direcciones lógicas con las físicas. Posee un número fijo de entradas y se utiliza para obtener la traducción rápida de direcciones. Si no existe una entrada buscada, se deberá revisar la tabla de paginación y tardará varios ciclos más, sobre todo si la página que contiene la dirección buscada no está en memoria primaria (memoria virtual). Si en la tabla de paginación no se encuentra la dirección buscada, saltará una interrupción conocida como fallo de página. Podemos almacenar la tabla de páginas en el propio traductor o en memoria principal:



- **En el traductor:** El TLB dispone de una zona en la que se almacena la tabla de páginas. En tiempo de conmutación, se tiene que copiar la tabla de páginas a memoria principal y se restaura la tabla de página del nuevo proceso que pasa a estado activo, lo cual ralentiza el tiempo de conmutación de procesos, lo que hace que no sea viable en sistemas modernos en los que se gestionan grandes cantidades de memoria.
- **En memoria principal:** El TLB dispone de unos registros que almacenan la dirección de memoria en la que se almacena la tabla de páginas y su tamaño. Ya que la tabla de páginas está en la memoria principal se necesita (por cada acceso a memoria) un acceso a memoria adicional para obtener el número de página física asociado a la página lógica. Para mantener un tiempo de acceso aceptable se establece una memoria asociativa en el TLB paginada que conserva la relación página lógica <-> página física utilizadas recientemente.

Paginación: Gestión

- **¿Cuál es el tamaño de página idóneo?**: Los valores óptimos para las porciones son en arquitectura de 32 bits es de 4 KiB, y en arquitectura de 64 bits es de 8 KiB.
- Los criterios vistos en la gestión de memoria mediante particiones de tamaño variable no tienen aplicación en la gestión de la memoria paginada, pues todos los huecos disponibles son de igual tamaño.

Un proceso dispone de todo el direccionamiento lógico que puede emplear según sus necesidades. En un procesador de 32 bits, un proceso dispone de $2^{32} = 4$ GiB de memoria lógica. Suponiendo que el tamaño de página es de 8 KiB (2^{13} bytes), en este procesador se dispone de un total de:

$$\frac{2^{32}}{2^{13}} = 2^{19} \text{ marcos} = 524288 \text{ marcos}$$

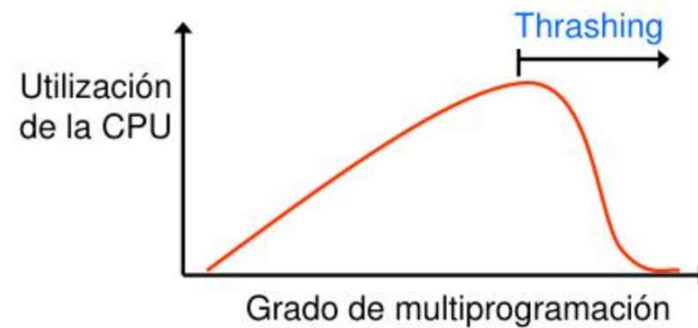
Para mantener la contabilidad de marcos libres se pueden emplear estructuras de datos tales como:

- **Mapa de bits de marcos libres**: Que contiene 1 si está asignada a un proceso, o 0 si está libre.
- **Lista de marcos libres**: Hay una lista global de marcos libres y una lista por proceso de marcos ocupados.

Al usar páginas de 4 KiB, todas las ubicaciones dentro de una página pueden ser representadas por 12 bits ($2^{12}=4096$), lo que deja 4 bits para representar el número de página. Si las páginas hubieran sido de la mitad del tamaño (2048) se podrían tener 5 bits para el número de página, lo que significa que a menor tamaño de página se pueden tener tablas con más páginas.

Hiperpaginación (Thrashing)

Un proceso genera fallos de página frecuentemente y el sistema pasa la mayor parte del tiempo paginando.



Posibles causas: Un proceso necesita de más marcos, su tasa de fallos de página aumenta y se produce la siguiente reacción en cadena:

- Disminuye el uso de la CPU
- El SO decide aumentar el grado de multiprogramación
- La tasa de fallos de página de incrementa

SOLUCIONES:

- Reducir la multiprogramación o emplear un algoritmo de reemplazo local o por prioridades
- Prevenir la hiperpaginación.

Paginación multinivel

- El objetivo es paginar la tabla de páginas lo que permite que esta no esté cargada completa en memoria y que no ocupe direcciones consecutivas.
- En los SO con tablas de páginas multinivel los números de página se dividen en dos partes: los bits más significativos indican el directorio de páginas correspondiente y con los bits menos significativos el índice del directorio en el que se encuentra la página buscada:

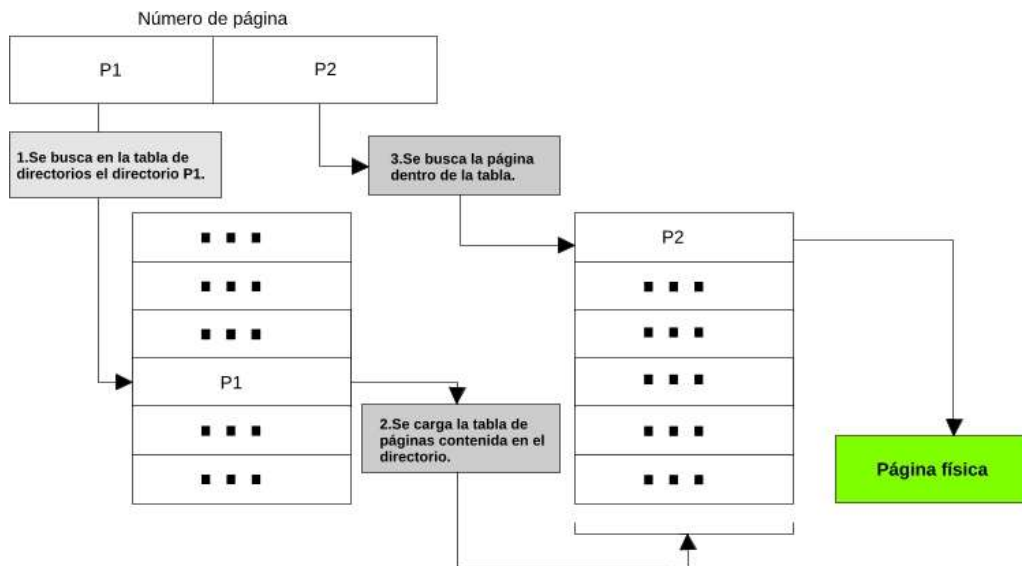
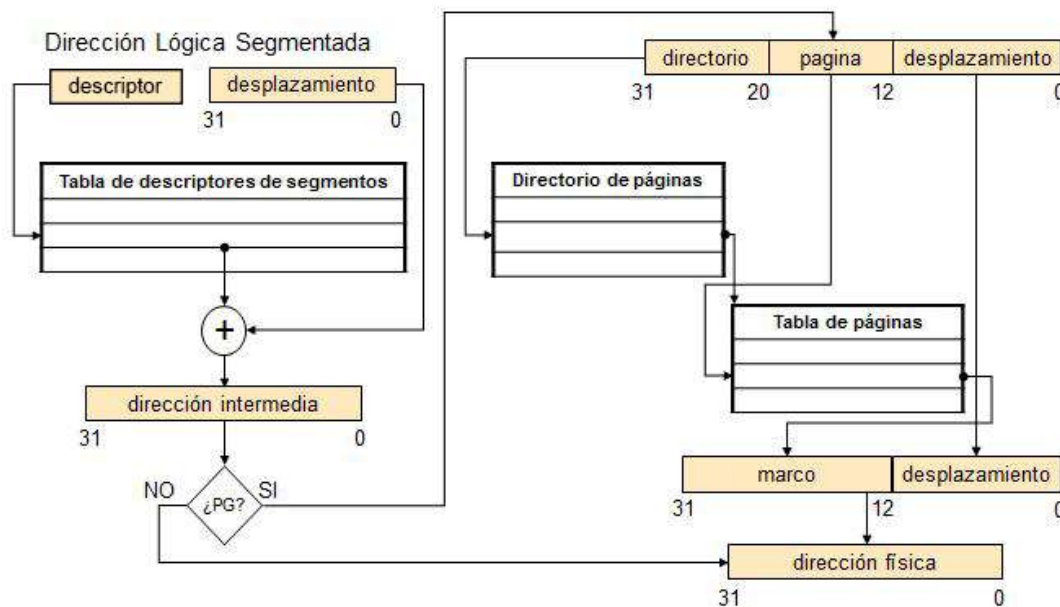


Tabla de páginas para todo el sistema:

Es una variante de las tablas de página multinivel, en la que el dispositivo traductor sólo contiene una entrada por cada proceso (indexadas por PID) y una dirección de la tabla que contiene las páginas de dicho proceso. Así, ante una conmutación de procesos basta con cambiar de PID.

Sistema combinado

Consiste en combinar la segmentación y la paginación. La **paginación segmentada** (paginación segmentando la tabla de paginas) tiene poco sentido practico y apenas se usa. En la **segmentación paginada**, la idea fundamental consiste en definir segmentos que están compuestos por N páginas. Por tanto, cualquier segmento ocupa, como mínimo una página de memoria.



Segmentación paginada en la arquitectura IA32

- Por una parte, la segmentación proporciona ventajas hacia los procesos: organización de datos y código, mecanismos de protección, compartición de datos y código y optimización en el espacio ocupado. Por otra parte, la paginación facilita la gestión de la memoria disponible.
- Los procesos emplean direcciones lógicas de memoria segmentada, que son traducidas a direcciones lógicas paginadas (dirección intermedia), que a su vez deben de ser traducidas a direcciones físicas. Por tanto, cualquier acceso a memoria supone un total de tres accesos a memoria. Uno para acceder a la tabla de segmentos, otro para acceder a la tabla de páginas y por último, otro más para acceder a la dirección física.